

석사학위논문
Master's Thesis

Ghidra P-code 기반 아키텍처 독립적 큐브셋 펌웨어
에뮬레이터

Ghidra P-code based architecture-independent CubeSat firmware
emulator

2026

류수현 (柳守炫 Ryu, Suhyeon)

한국과학기술원

Korea Advanced Institute of Science and Technology

석사학위논문

Ghidra P-code 기반 아키텍처 독립적 큐브셋 펌웨어
에뮬레이터

2026

류수현

한국과학기술원

전산학부

Ghidra P-code 기반 아키텍처 독립적 큐브셋 펌웨어 에뮬레이터

류 수 현

위 논문은 한국과학기술원 석사학위논문으로
학위논문 심사위원회의 심사를 통과하였음

2026년 6월 10일

심사위원장 류 석 영 (인)

심 사 위 원 유 신 (인)

심 사 위 원 홍 신 (인)

Ghidra P-code based architecture-independent CubeSat firmware emulator

Suhyeon Ryu

Advisor: Sukyoung Ryu

A dissertation submitted to the faculty of
Korea Advanced Institute of Science and Technology in
partial fulfillment of the requirements for the degree of
Master of Science in Computer Science

Daejeon, Korea
June 10, 2026

Approved by

Sukyoung Ryu
Professor of Computer Science

The study was conducted in accordance with Code of Research Ethics¹.

¹ Declaration of Ethical Conduct in Research: I, as a graduate student of Korea Advanced Institute of Science and Technology, hereby declare that I have not committed any act that may damage the credibility of my research. This includes, but is not limited to, falsification, thesis written by someone else, distortion of research findings, and plagiarism. I confirm that my thesis contains honest conclusions based on my own careful research under the guidance of my advisor.

MCS 류수현. Ghidra P-code 기반 아키텍처 독립적 큐브셋 펌웨어 에뮬레이터. 전산학부 . 2026년. 26+iv 쪽. 지도교수: 류석영. (한글 논문)
Suhyeon Ryu. Ghidra P-code based architecture-independent CubeSat firmware emulator. School of Computing . 2026. 26+iv pages. Advisor: Sukyoung Ryu. (Text in Korean)

초 록

큐브셋 펌웨어는 주변장치와 밀접하게 결합되어 동작하므로, 바이너리만으로 수행하는 정적 분석은 실행 중 하드웨어 상태에 의존하는 경로를 놓칠 수 있다. 세부적으로는 타이머나 인터럽트 컨트롤러와 같은 주변장치들과 전파 송수신기와 같은 외부장치들이 포함되며, 이들 하드웨어들의 상태는 간접 제어 흐름의 목적지에 영향을 준다. 본 연구는 Ghidra P-code 기반 아키텍처 독립적 큐브셋 펌웨어 에뮬레이터를 제안한다. 제안한 에뮬레이터는 PcodeOp 단위에서 메모리 접근을 관찰하여 주변 및 외부 장치 모델과 연결, 관찰한 간접 제어 흐름을 Ghidra 참조와 북마크로 기록한다. RANDEV OBC 펌웨어에 적용한 결과, 원격 측정 명령을 성공적으로 처리하였고 도달 가능 함수 비율을 78.0%에서 83.1%로 높였으며, 간접 제어 흐름 위치 58개 중 16개의 실제 대상을 확인하였다. 이는 전체 하드웨어 접근 없이도 주변장치 의존 펌웨어의 정적 분석을 보완할 수 있음을 보인다.

핵심 낱말 큐브셋 펌웨어, 에뮬레이션, Ghidra, P-code, 정적 분석

Abstract

CubeSat firmware operates in close interaction with peripheral devices, and thus static analysis performed only on a binary can miss paths that depend on hardware states during execution. Specifically, these hardware components include peripherals such as timers and interrupt controllers and external devices such as radio transceivers, and their states affect the destinations of indirect control flows. This thesis proposes a Ghidra P-code-based architecture-independent CubeSat firmware emulator. The proposed emulator observes memory accesses at the PcodeOp level, connects them to peripheral and external device models, and records the observed indirect control flows as Ghidra references and bookmarks. When applied to the RANDEV OBC firmware, the emulator successfully processed telemetry commands, increased the reachable function ratio from 78.0% to 83.1%, and identified concrete targets for 16 out of 58 indirect control-flow sites. This shows that static analysis of peripheral-dependent firmware can be complemented even without access to the full hardware.

Keywords CubeSat firmware, emulation, Ghidra, P-code, static analysis

차 례

차 례	i
표 차례	iii
그림 차례	iv
제 1 장 머리말	1
제 2 장 연구 배경	2
2.1 큐브셋 펌웨어의 특징	2
2.2 RANDEV OBC 펌웨어	2
2.2.1 구조 개요	3
2.2.2 통신 흐름	3
2.2.3 작업 및 인터럽트 흐름	4
2.3 Ghidra	4
2.3.1 Ghidra 스크립트	5
2.3.2 Sleigh 명세 언어	5
2.3.3 P-code와 PcodeOp	5
제 3 장 접근 방법	7
3.1 기본 Ghidra 에뮬레이터의 한계	7
3.2 본 연구의 접근	7
3.3 적용 대상과 범위	8
제 4 장 구현	9
4.1 개요	9
4.2 Ghidra 스크립트로 PcodeOp 단위 실행	9
4.3 펌웨어 초기 실행 환경 구성	10
4.4 P-code 후킹을 통한 MMIO 장치 연결	10
4.5 타이머 및 인터럽트	11
4.6 사용자 연산(userop) 처리	11
4.7 I2C 장치 연결	12
4.8 TCP 클라이언트를 통한 RF 통신	12
4.9 동적 제어 흐름 기록	13
제 5 장 실험 및 결과	14
5.1 실험 환경	14

5.2	펌웨어 실행 및 원격 측정 응답 검증	14
5.3	Ghidra 프로그램 메타데이터 보강	15
5.4	확인된 불일치 사례	16
제 6 장	논의 및 향후 연구 방향	18
6.1	다른 펌웨어로의 확장 가능성	18
6.2	실행 속도 한계	18
6.3	향후 연구 방향	19
제 7 장	관련 연구	21
7.1	펌웨어 에뮬레이션	21
7.2	위성 및 큐브셋 소프트웨어 검증	21
7.3	정적 분석 및 Ghidra 기반 분석	21
제 8 장	맺음말	23
참 고 문 헌		24
사 사		25
약 력		26

표 차례

5.1	원격 측정 명령 응답 비교 결과	15
5.2	에뮬레이션 전후 도달 가능 함수 변화	16
5.3	간접 제어 흐름 대상 확인 결과	16
6.1	QEMU와 Ghidra 기반 에뮬레이터의 실행 시간 비교	19

그림 차례

2.1	RANDEV OBC와 주요 서브시스템 간 통신 구조	3
2.2	지상국과 RANDEV OBC 사이의 데이터 통신 흐름	4
4.1	RANDEV 주변장치 모델 구조도 (추가 예정)	11

제 1 장 머리말

최근 위성 기술은 지구 관측, 과학 실험, 통신, 항법 등 다양한 분야의 수요 증가에 따라 빠르게 발전하고 있다. 위성은 현대 사회의 핵심 기반 시설 중 하나로 자리 잡았으며, 환경 감시와 재난 대응부터 원격 통신 및 위치 기반 서비스에 이르기까지 다양한 응용을 가능하게 한다. 특히 우주급 하드웨어의 소형화와 상용 부품 생태계의 성장은 대형 고비용 위성뿐 아니라 작고 비용 효율적인 위성 체계의 활용 가능성을 크게 넓혔다.

이러한 흐름 속에서 큐브셋(CubeSat)은 우주 접근의 진입 장벽을 낮춘 대표적인 소형 위성 규격으로 활용되고 있다. 큐브셋은 10cm 정육면체를 기본 단위인 1유닛(1U)으로 사용하는 표준화된 소형 위성 규격이며[1], 모듈화된 구조를 바탕으로 교육, 연구, 기술 실증, 상업 임무에 폭넓게 사용된다. 작은 크기와 낮은 개발 비용은 대학 연구실, 학생 팀, 신생 기업과 같은 비교적 작은 조직도 실제 우주 임무에 참여할 수 있도록 하였고, 이는 큐브셋 발사 및 운용 사례의 증가로 이어지고 있다.

그러나 큐브셋의 높은 접근성은 동시에 검증 측면의 어려움을 수반한다. 큐브셋은 제한된 예산, 짧은 개발 기간, 한정된 인력 조건에서 설계 및 시험되는 경우가 많으며, 실제 하드웨어를 포함한 반복적인 검증 환경을 충분히 확보하기 어렵다. 반면 궤도 환경에서 동작하는 위성은 발사 이후 수정이 제한적이고, 온보드 컴퓨터(on-board computer, OBC) 펌웨어의 결함은 통신 두절, 자세 제어 실패, 전력 관리 오류와 같은 임무 실패로 이어질 수 있다. 따라서 발사 이전 단계에서 OBC 펌웨어를 가능한 한 넓게 분석하고 검증하는 과정이 중요하다.

펌웨어 분석에는 Ghidra[3]와 같은 역공학 도구를 사용할 수 있다. Ghidra는 바이너리 적재, 역어셈블, 역컴파일, 참조 및 호출 그래프 생성 등 정적 분석에 필요한 다양한 기능을 제공한다. 하지만 큐브셋 펌웨어는 단순히 중앙 처리 장치(CPU) 명령어만 실행하는 프로그램이 아니라, 주변 및 외부 장치들과 지속적으로 상호작용하는 시스템 소프트웨어이다. 따라서 이러한 하드웨어의 실행 환경 없이 정적 분석만 수행할 경우 실제 실행 중에만 드러나는 제어 흐름을 복원하기 어렵다. 예를 들어 타이머 값이나 장치 상태가 특정 조건을 만족해야 진행되는 분기, 외부 장치 응답에 따라 선택되는 명령 처리 경로는 순수 정적 분석만으로는 확인하기 어렵다.

이러한 문제를 해결하기 위해 본 연구에서는 Ghidra P-code 기반의 큐브셋 펌웨어 에뮬레이터를 구현하였다. 제안하는 에뮬레이터는 Ghidra의 P-code 실행 엔진을 기반으로 하며, PcodeOp 단위의 실행 후킹을 통해 주변장치들이 모델링된 펌웨어 실행 환경과 상호작용한다. 이를 통해 실제 하드웨어나 전체 소스 코드가 없는 상황에서도 Ghidra에 적재된 펌웨어 바이너리를 실행하고, 실행 중 관찰된 동적 제어 흐름을 Ghidra 프로그램 메타데이터에 반영할 수 있도록 하였다.

본 연구는 RANDEV OBC 펌웨어를 대상으로 제안한 에뮬레이터를 구현하고 평가하였다. 구현 결과, 펌웨어는 에뮬레이터 위에서 원격 측정 명령을 처리하고 무선 통신 모델을 통해 응답을 반환할 수 있었으며, 실행 과정에서 확인된 간접 제어 흐름 대상을 Ghidra 참조와 북마크로 기록하여 기존 정적 분석 결과를 보강할 수 있었다. 본 연구의 주요 기여는 다음과 같다.

1. Ghidra P-code 실행 엔진을 기반으로 하는 아키텍처 독립적 큐브셋 펌웨어 에뮬레이션 프레임워크를 제안하고 구현하였다.
2. 큐브셋 펌웨어 실행에 필요한 주변 및 외부 장치 모델을 모듈화하여 펌웨어 바이너리 실행을 위한 하드웨어 환경을 구성하였다.
3. 에뮬레이션 중 관찰된 간접 제어 흐름을 Ghidra 프로그램 메타데이터에 반영하여 정적 분석 범위를 확장할 수 있음을 보였다.

제 2 장 연구 배경

2.1 큐브셋 펌웨어의 특징

큐브셋은 하나의 단일 장치가 아니라 여러 서브시스템이 결합된 임베디드 시스템으로 동작한다. 일반적인 큐브셋은 임무와 설계에 따라 세부 구성이 달라질 수 있지만, 대체로 다음과 같은 서브시스템을 포함한다.

- OBC 서브시스템: 위성의 중심 제어를 담당하며, 명령 처리, 원격 측정 데이터 수집, 작업 스케줄링, 다른 서브시스템과의 통신을 수행한다. OBC 펌웨어는 이 서브시스템에서 실행된다.
- 전력계(Electrical Power System, EPS): 태양전지, 배터리, 전력 분배 회로 등을 관리하며, 전압, 전류, 배터리 상태와 같은 전력 관련 상태 정보를 제공한다.
- 자세 제어계(Attitude Determination and Control System, ADCS): 위성의 자세를 추정하고 제어하기 위한 센서와 구동기를 관리하며, 자세 상태와 제어 명령을 처리한다.
- 통신계: 지상국 또는 외부 통신 상대와의 무선 송수신을 담당하며, 외부 명령 수신과 원격 측정 데이터 송신에 사용된다.
- 탑재체 및 기타 장치: 임무 수행을 위한 센서, 저장장치, 안테나 관련 장치 등으로 구성되며, 임무 목적에 따라 구성이 달라진다.

큐브셋 펌웨어는 OBC에서 실행되며, 위성의 명령 처리, 원격 측정 수집, 주변장치 제어, 주기적 상태 점검과 같은 핵심 동작을 담당한다. 본 연구에서 중요한 점은 큐브셋의 임무나 활용 분야보다는, OBC 펌웨어가 OBC 내부 주변장치 및 외부 서브시스템과 밀접하게 결합된 임베디드 시스템 소프트웨어라는 점이다.

이러한 펌웨어는 주변 및 외부 장치들을 포함한 하드웨어 실행 환경에 강하게 의존한다. 따라서 펌웨어 바이너리를 분석할 때 중앙 처리 장치 명령어만 실행하거나 정적 분석만 수행하면, 실제 임무 수행 중 외부 장치 상태에 의해 결정되는 제어 흐름을 충분히 복원하기 어렵다. 본 연구의 에뮬레이터는 이러한 특성을 반영하기 위해 펌웨어가 관찰하는 레지스터 값, 인터럽트, 패킷, 명령 응답과 같은 관찰 가능한 동작을 모델링한다.

2.2 RANDEV OBC 펌웨어

본 연구에서는 RANDEV OBC 펌웨어를 분석 및 에뮬레이션 대상으로 사용하였다. RANDEV OBC 펌웨어는 AVR32 기반 환경에서 동작하며, 여러 주변장치와 통신하면서 외부 명령을 처리하고 원격 측정 데이터를 생성한다. 본 연구에서 RANDEV를 대상으로 선택한 이유는 MMIO, 타이머, 인터럽트, DMA, I2C, 무선 통신 등 큐브셋 펌웨어 에뮬레이션에 필요한 주요 요소가 함께 나타나며, 기존 실행 결과와 일부 소스 코드를 참조하여 에뮬레이터의 동작을 검증할 수 있기 때문이다.

또한 RANDEV 개발자로부터 펌웨어의 소스 코드를 제공받을 수 있어, 주변장치 통신 규칙과 기존 실행 결과를 확인하며 에뮬레이터 구현의 기준으로 활용할 수 있었다. 다만 본 연구의 에뮬레이션 대상은 Ghidra에 가져온 펌웨어 바이너리이며, 소스 코드는 장치 모델의 동작과 실행 결과를 검증하기 위한 보조 자료로 사용하였다.

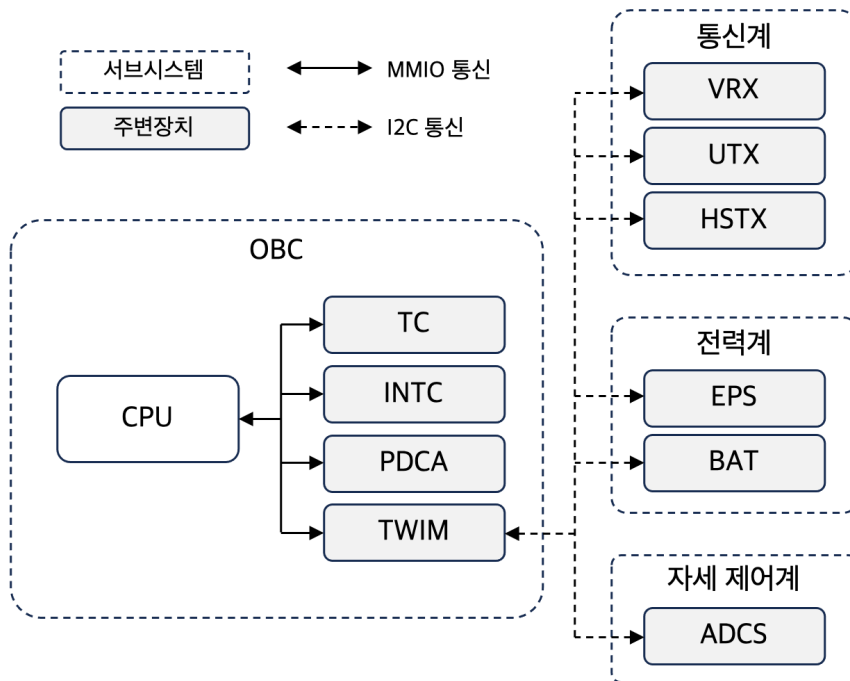


그림 2.1: RANDEV OBC와 주요 서브시스템 간 통신 구조

2.2.1 구조 개요

그림 2.1은 RANDEV OBC와 주요 서브시스템 간 통신 구조를 보여준다. OBC 내부에는 CPU와 함께 TC, INTC, PDCA, TWIM과 같은 내부 주변장치가 존재하며, CPU는 이들을 MMIO 레지스터 접근을 통해 제어한다. 반면 전력계, 자세 제어계, 통신계에 속한 EPS, BAT, ADCS, VRX, UTX, HSTX 등은 OBC 외부의 서브시스템 또는 장치로, OBC는 TWIM을 통해 이들과 I2C 방식으로 통신한다. 그림의 실선 화살표는 OBC 내부에서 이루어지는 MMIO 기반 접근을, 점선 화살표는 외부 장치와의 I2C 통신 경로를 나타낸다. 따라서 RANDEV OBC 펌웨어의 실행 상태는 CPU 내부 상태뿐 아니라 MMIO로 접근되는 내부 주변장치 상태, 외부 I2C 장치의 응답, 무선 패킷 큐와 같은 외부 실행 환경에 의해 함께 결정된다.

2.2.2 통신 흐름

RANDEV OBC 펌웨어의 명령 및 원격 측정 처리는 RF 통신, I2C 통신, DMA 전송, MMIO 레지스터 접근이 결합된 형태로 이루어진다. 그림 2.2은 지상국과 RANDEV OBC 사이에서 명령 및 원격 측정 데이터가 이동하는 주요 경로를 단순화하여 나타낸 것이다. 그림에서 박스는 데이터가 저장되거나 펌웨어가 관찰하는 지점을 나타내며, 하단의 라벨은 각 구간의 전송 방식 및 전송을 담당하는 OBC 내부 주변장치를 나타낸다.

외부에서 전달된 RF 패킷은 통신계의 수신 장치인 VRX에 도달한다. OBC 펌웨어가 TWIM을 통해 VRX로부터 데이터를 읽으면, 해당 데이터는 TWIM의 수신 보관 레지스터(receive holding register, RHR)에 저장된다. 이후 PDCA 기반 DMA 전송을 통해 RHR의 데이터가 OBC 메모리로 복사되고, 펌웨어는 메모리에 저장된 패킷을 해석하여 명령을 처리한다.

반대 방향의 원격 측정 응답은 OBC 메모리에서 시작된다. 펌웨어가 응답 프레임을 구성하면 PDCA는 메모리의 데이터를 TWIM의 송신 보관 레지스터(transmit holding register, THR)로 전송하고, TWIM

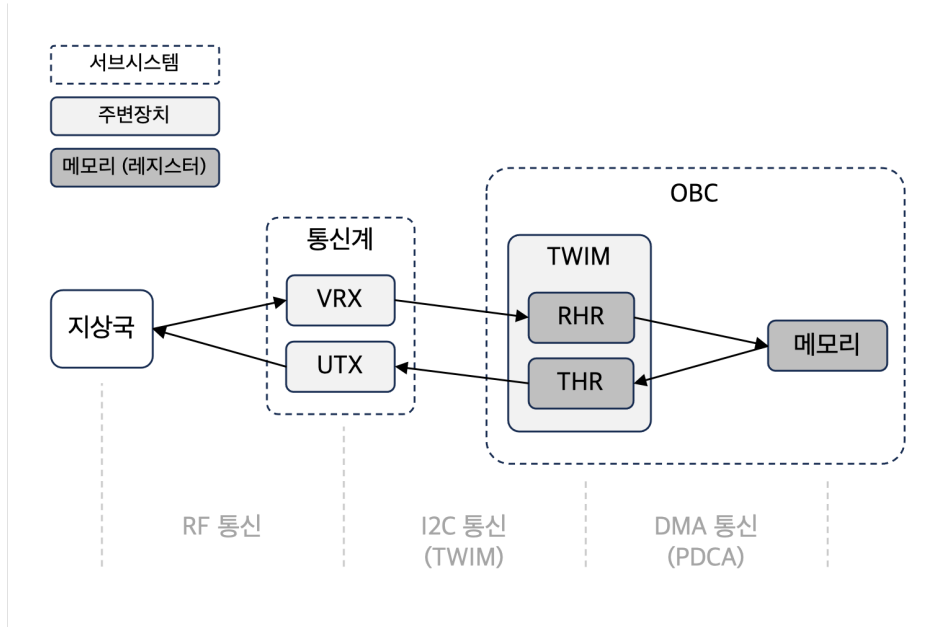


그림 2.2: 지상국과 RANDEV OBC 사이의 데이터 통신 흐름

은 이를 I2C 통신을 통해 UTX로 전달한다. UTX는 전달받은 프레임을 RF 통신을 통해 지상국으로 송신한다. 즉 VRX는 지상국에서 OBC로 들어오는 데이터 경로를, UTX는 OBC에서 지상국으로 나가는 데이터 경로를 담당한다.

그림은 주요 데이터 이동 방향을 중심으로 나타낸 것이며, 실제 I2C 통신에서는 OBC 펌웨어가 TWIM 및 PDCA를 설정하여 각 장치에 명령을 보내고 응답을 읽어 온다. 이러한 통신 흐름은 이후 RF 패킷 큐, TWIM의 RHR/THR 상태, PDCA 전송, I2C 장치 응답 모델링의 구현 배경이 된다.

2.2.3 작업 및 인터럽트 흐름

RANDEV OBC 펌웨어는 타이머 인터럽트를 기반으로 주기적인 작업 문맥 전환을 수행한다. TC는 설정된 클럭과 비교 조건에 따라 인터럽트를 발생시키며, INTC는 각 인터럽트의 대기 상태와 우선순위를 관리한다. 펌웨어는 인터럽트 핸들러(interrupt handler)에 진입한 뒤 스케줄러 관련 처리를 수행하고, 필요에 따라 현재 작업의 문맥을 저장하거나 다른 작업으로 실행 흐름을 전환한다.

이와 같은 작업 및 인터럽트 흐름은 이후 TC와 INTC 모델의 세부 구현에서 배경이 된다.

2.3 Ghidra

Ghidra는 바이너리 프로그램의 역공학을 지원하는 소프트웨어 분석 프레임워크이다[3]. Ghidra는 바이너리 가져오기, 역어셈블, 역컴파일, 심볼 및 참조 분석, 함수 생성과 호출 그래프 분석 등 펌웨어 정적 분석에 필요한 기능을 제공한다. 본 연구에서는 Ghidra를 단순한 역어셈블러가 아니라, 펌웨어 실행 결과를 다시 프로그램 메타데이터로 반영할 수 있는 분석 환경으로 사용한다.

Ghidra를 본 연구의 기반으로 사용한 이유는 명령어 의미가 P-code라는 아키텍처 독립적인 중간 표현으로 변환되기 때문이다. 이를 활용하면 특정 아키텍처의 명령어 실행기를 새로 구현하지 않고도, Ghidra가 제공하는 P-code 실행 엔진 위에서 펌웨어를 실행할 수 있다. 또한 실행 중 관찰된 간접 제어 흐름, 참조, 북마크 등의 정보를 Ghidra 프로그램에 기록함으로써 정적 분석 결과를 보강할 수 있다.

2.3.1 Ghidra 스크립트

Ghidra 스크립트는 Ghidra 내부의 프로그램 데이터베이스와 분석 기능을 자동화하기 위한 인터페이스이다. 스크립트를 통해 프로그램 메모리, 레지스터, 심볼, 함수, 참조, 북마크 등에 접근할 수 있으며, 그래픽 사용자 인터페이스 환경뿐 아니라 Headless Ghidra 환경에서도 대상 프로그램을 대상으로 실행할 수 있다. 본 연구에서는 Headless Ghidra에서 스크립트를 실행하여 펌웨어 에뮬레이션, 주변장치 모델 연결, 동적 제어 흐름 기록, 메타데이터 갱신을 수행하였다.

2.3.2 Sleigh 명세 언어

Sleigh는 Ghidra가 프로세서 아키텍처의 명령어 형식과 의미를 정의하기 위해 사용하는 명세 언어이다[4]. 각 아키텍처의 Sleigh 명세는 바이너리 명령어를 해석하고, 해당 명령어가 어떤 PcodeOp으로 변환되는지를 정의한다. 따라서 Ghidra가 특정 아키텍처의 Sleigh 명세를 제공한다면, 본 연구의 에뮬레이터는 해당 명령어를 직접 해석하지 않고도 P-code 단위 실행 구조를 재사용할 수 있다.

예를 들어 AVR32의 ICALL(indirect call) 명령어는 Ghidra의 Sleigh 명세에서 다음과 같이 정의된다.¹

```
# ICALL Format I:
# Operation:  LR <- PC + 2
#             PC <- Rd
# Syntax:    icall Rd
# 0101 1101 0001 dddd

:ICALL rd0 is op4_12=0x5d1 & rd0 {
    tmp:4 = rd0;
    LR = inst_next;
    call [tmp];
}
```

위 명세에서 앞부분은 `icall Rd` 명령어의 비트 패턴과 피연산자를 정의하고, 중괄호 내부는 해당 명령어의 의미를 Sleigh의 P-code 구문으로 표현한다. `LR = inst_next`는 다음 명령어 주소를 링크 레지스터에 저장하는 동작을 나타내며, `call [tmp]`는 `rd0` 레지스터 값으로 이동하는 간접 호출을 나타낸다. 이처럼 Sleigh 명세는 아키텍처별 명령어를 Ghidra가 공통적으로 실행하고 분석할 수 있는 중간 표현으로 연결한다.

2.3.3 P-code와 PcodeOp

P-code는 Ghidra의 중간 표현으로, 아키텍처 의존적인 명령어 의미를 일반화된 연산들의 열로 표현한다. 하나의 기계어 명령어는 여러 개의 PcodeOp으로 나뉘 수 있으며, PcodeOp은 Ghidra 에뮬레이터가 순차적으로 실행하는 기본 연산 단위이다.

앞 절의 ICALL 명세는 명령어의 의미를 Sleigh 구문으로 표현한 것이다. 실제로 RANDEV OBC 펌웨어의 `service_dispatcher_task` 함수에 있는 ICALL R0 명령어를 Ghidra 스크립트로 확인하면 다음과 같은 PcodeOp으로 변환된다.

```
address: 8001f4a6
function: service_dispatcher_task
instruction: ICALL R0
```

¹https://github.com/NationalSecurityAgency/ghidra/blob/master/Ghidra/Processors/Atmel/data/languages/avr32a_data_transfer.sinc

pcode:

```
(register, 0x1038, 4) COPY (const, 0x8001f4a8, 4)
--- CALLIND (register, 0x1000, 4)
```

위 출력에서 COPY는 다음 명령어 주소를 링크 레지스터에 저장하는 동작에 해당하고, CALLIND는 R0에 저장된 주소로 이동하는 간접 호출에 해당한다. 이는 앞 절의 Sleigh 명세에서 LR = inst_next와 call [tmp]로 표현된 동작과 같은 의미이다. 이처럼 Ghidra는 아키텍처별 명령어를 PcodeOp의 옰로 변환하여 실행한다.

이처럼 P-code를 사용하면 서로 다른 아키텍처의 명령어도 공통된 실행 단위로 관찰할 수 있다. 본 연구는 이러한 특성을 이용하여 3장에서 설명할 PcodeOp 단위 후킹과 Ghidra 메타데이터 반영을 수행한다.

제 3 장 접근 방법

이 장에서는 큐브셋 펌웨어를 Ghidra 환경에서 실행하고, 그 실행 결과를 정적 분석에 활용하기 위한 본 연구의 접근 방법을 설명한다. 앞 장에서 설명한 것처럼 RANDEV OBC 펌웨어는 MMIO, TC/INTC, DMA, I2C, 무선 통신과 같은 주변장치 상태에 의존한다. 따라서 단순히 중앙 처리 장치 명령어를 실행하는 것만으로는 펌웨어의 실제 동작을 충분히 관찰하기 어렵다.

3.1 기본 Ghidra 에뮬레이터의 한계

Ghidra는 프로그램 분석을 위한 에뮬레이션 기능을 제공하지만, 기본 에뮬레이터만으로 큐브셋 펌웨어의 전체 실행 환경이 자동으로 구성되는 것은 아니다. 기본 에뮬레이터는 명령어 의미를 실행할 수 있으나, 펌웨어가 접근하는 주변장치의 레지스터 상태와 부수 효과를 알지 못한다. 따라서 일반 메모리와 달리 특정 주소 범위에 대한 적재/저장 연산이 실제 장치 동작으로 이어져야 하는 베어메탈 펌웨어에서는 추가적인 환경 모델링이 필요하다.

큐브셋 펌웨어 실행에서 기본 에뮬레이터만으로 처리하기 어려운 요소는 다음과 같다.

- MMIO 레지스터 접근 시 주변장치의 상태 변화나 인터럽트 플래그 설정과 같은 부수 효과가 발생하지 않는다.
- TC의 카운터 값이 실제로 증가하지 않기 때문에 틱 인터럽트와 주기적 작업 실행이 재현되지 않는다.
- INTC가 대기 중인 인터럽트, 우선순위, 마스크 상태를 관리하지 않으므로 인터럽트 핸들러 진입과 문맥 전환이 일어나지 않는다.
- DMA 컨트롤러와 I2C 컨트롤러의 상태가 모델링되지 않아 메모리와 I2C 장치 사이의 명령-응답 통신이 진행되지 않는다.
- 무선 패킷 큐가 존재하지 않아 외부 명령 입력과 원격 측정 송신 흐름을 재현하기 어렵다.

이러한 한계는 정적 분석 결과에도 영향을 준다. 예를 들어 타이머 값이나 장치 상태가 특정 조건을 만족해야 통과하는 분기는 실행되지 않으며, I2C 장치 응답에 따라 호출되는 명령 핸들러도 관찰되지 않는다. 또한 함수 포인터나 분기 테이블을 통해 수행되는 간접 호출의 대상은 실행 중 확인될 수 있음에도, 기본 에뮬레이션 결과가 Ghidra 참조나 호출 그래프에 자동으로 반영되지는 않는다. 따라서 큐브셋 펌웨어 분석을 위해서는 실행 환경 모델링과 메타데이터 갱신이 함께 필요하다.

3.2 본 연구의 접근

본 연구는 새로운 중앙 처리 장치 에뮬레이터를 직접 구현하지 않고, Ghidra가 제공하는 P-code 실행 엔진을 재사용한다. 명령어 해석과 의미 변환은 Ghidra의 Sleight 명세와 P-code 실행 체계를 통해 해결하고, 본 연구에서는 큐브셋 펌웨어 실행에 필요한 주변장치 환경과 분석 결과 반영 계층을 추가한다. 이를 통해 특정 아키텍처의 명령어 집합을 직접 구현하지 않으면서도, 펌웨어 실행 과정에서 필요한 지점에 분석 목적의 후킹을 삽입할 수 있다.

본 연구의 핵심 접근은 PcodeOp 단위 실행을 관찰하는 것이다. 기계어 명령어는 Ghidra 내부에서 하나 이상의 PcodeOp으로 변환되므로, PcodeOp 단위로 실행을 제어하면 명령어 내부에서 발생하는 메모리 접근, 분기, 사용자 연산을 세밀하게 확인할 수 있다. 본 연구에서는 다음과 같은 방식으로 PcodeOp을 활용한다.

- LOAD, STORE를 관찰하여 MMIO 주소 접근을 감지하고, 해당 접근을 장치 모델의 읽기/쓰기 동작으로 전달한다.
- CALLIND, BRANCHIND, RETURN 등 제어 흐름 관련 PcodeOp을 관찰하여 실행 중 확인된 동적 대상을 기록한다.
- CALLOTHER로 표현되는 사용자 연산을 처리하여 기본 Ghidra 실행 엔진만으로 수행되지 않는 아키텍처 의존 동작을 보완한다.

주변장치 모델은 펌웨어가 관찰하는 동작을 중심으로 구성한다. MMIO 접근은 주소 범위를 기준으로 장치 관리자(DeviceManager)에 전달되며, 장치 관리자는 접근 주소가 어느 장치와 레지스터에 해당하는지를 판별한다. 각 장치 모델은 레지스터 읽기/쓰기, 인터럽트 발생, DMA 전송, I2C 명령-응답, 무선 패킷 큐와 같은 동작을 구현한다. 이 방식은 실제 하드웨어의 모든 내부 상태보다 펌웨어가 실행 중 관찰하는 동작을 우선한다.

이때 본 연구에서 사용하는 MMIO 장치와 I2C 장치라는 구분은 실제 위성 서브시스템의 물리적 계층을 그대로 반영한 것이 아니라, 펌웨어가 해당 대상을 어떤 방식으로 접근하는지를 기준으로 한 에뮬레이터 모델링상의 구분이다. TC, INTC, PDCA, TWIM은 OBC 내부에서 레지스터 접근을 통해 제어되므로 MMIO 장치로 모델링하였고, EPS, VRX, UTX, HSTX, ADCS, UVANT 등은 TWIM/PDCA를 거친 명령-응답 방식으로 접근되므로 I2C 장치로 모델링하였다. 즉 본 연구의 장치 구분은 실제 위성의 서브시스템 분류가 아니라, 펌웨어 관점에서 관찰되는 접근 경로를 기준으로 한다.

또한 본 연구는 에뮬레이션 결과를 Ghidra 프로그램 메타데이터에 반영한다. 실행 중 관찰된 간접 제어 흐름 대상은 참조와 북마크로 기록되며, 이후 호출 그래프나 도달 가능 함수 분석에서 기존 정적 분석만으로는 확인되지 않던 간선을 포함할 수 있다. 즉 본 연구의 에뮬레이터는 단순히 펌웨어를 실행하는 도구가 아니라, Ghidra 정적 분석의 범위를 확장하기 위한 실행 기반 분석 계층으로 동작한다.

3.3 적용 대상과 범위

본 연구의 적용 대상은 Ghidra에 가져온 RANDEV OBC 펌웨어이다. 평가 대상 펌웨어는 AVR32 기반이지만, 본 연구의 실행 구조는 Ghidra P-code를 기준으로 구성되어 있으므로 명령어 실행 방식 자체는 특정 아키텍처에 직접 의존하지 않는다. 다만 RANDEV의 주변장치 주소 맵, 레지스터 부수 효과, 인터럽트 구조, I2C 통신 규약, 무선 패킷 형식과 같은 정보는 대상 의존적으로 모델링하였다.

본 연구가 목표로 하는 것은 주기 정확 하드웨어 시뮬레이션이 아니다. 대신 펌웨어가 실행 중 관찰하는 레지스터 값, 인터럽트, 패킷, 명령 응답과 같은 관찰 가능한 동작을 제공하여, 실제 하드웨어 없이도 명령 처리, 원격 측정 응답, 작업/인터럽트 흐름을 재현하는 것을 목표로 한다. 따라서 전기적 신호, 센서의 물리 모델, 궤도 환경, 모든 가능한 외부 입력에 대한 전체 탐색은 본 연구의 범위에 포함하지 않는다.

본 연구에서 다루는 범위는 다음과 같이 정리할 수 있다.

- Ghidra P-code 기반 펌웨어 실행 및 PcodeOp 단위 관찰
- RANDEV 실행에 필요한 주변장치 및 외부 통신 모델링
- 실행 중 확인된 동적 제어 흐름의 Ghidra 메타데이터 반영

이 중 P-code 실행 구조, PcodeOp 후킹, 장치 추상화, 메타데이터 갱신 방식은 다른 펌웨어에도 재사용 가능한 부분이다. 반면 MMIO 기준 주소, 레지스터 배치, 인터럽트 라인, I2C 슬레이브 주소, 명령-응답 통신 규약, 일부 사용자 연산 처리 등은 대상에 따라 새로 정의되어야 한다. 따라서 본 연구의 범위는 특정 큐브셋 하드웨어를 완전히 일반화하는 것이 아니라, Ghidra 기반 분석 환경에서 주변장치 의존적인 펌웨어 실행을 재현하고 정적 분석을 보완할 수 있음을 보이는 데 있다.

제 4 장 구현

4.1 개요

본 장에서는 3장에서 설명한 접근 방법을 RANDEV OBC 펌웨어에 적용한 구현을 설명한다. 구현은 Headless Ghidra 환경에서 실행되는 Ghidra 스크립트 형태로 구성되며, 실행 시작 전 펌웨어 초기 상태와 장치 모델을 설정한 뒤 Ghidra P-code 실행 루프를 구동한다.

에뮬레이션의 전체 흐름은 다음과 같다. 먼저 스크립트는 Ghidra 프로그램에 적재된 RANDEV OBC 펌웨어를 대상으로 실행되며, 펌웨어 실행에 필요한 초기 레지스터와 메모리 상태를 설정한다. 이후 MMIO 장치와 I2C 장치를 초기화하고, TC, INTC, PDCA, TWIM, VRX, UTX 등 장치 사이의 연결 관계를 구성한다. 준비가 끝나면 진입점을 설정하고 P-code 실행 루프를 시작한다.

주 에뮬레이션 루프는 명령어 단위로 진행되지만, 각 명령어의 내부 실행은 PcodeOp 단위로 관찰된다. 실행 루프는 현재 명령어의 PcodeOp을 하나씩 실행하면서 메모리 접근, 시스템 레지스터 접근, 사용자 연산 등을 확인한다. 이 과정에서 MMIO 주소에 대한 LOAD 또는 STORE가 발견되면 해당 접근을 일반 메모리 접근으로 처리하지 않고 장치 모델로 전달한다. 명령어 실행이 끝난 뒤에는 대기 중인 인터럽트를 확인하여 인터럽트 진입 여부를 결정하고, 실행 후 프로그램 카운터를 바탕으로 간접 제어 흐름 대상을 기록한다.

TC 모델의 클럭 스트레드와 RF 모듈 시뮬레이터는 주 실행 루프와 별도로 동작한다. TC 클럭 스트레드는 타이머 카운터와 인터럽트 발생을 담당하며, RF 모듈 시뮬레이터는 TCP 클라이언트와 연결되어 외부 명령 패킷을 VRX 큐에 전달하거나 UTX 큐의 원격 측정 패킷을 외부로 송신한다.

4.2 Ghidra 스크립트로 PcodeOp 단위 실행

본 연구에서는 펌웨어 실행을 위해 Ghidra 스크립트를 사용하였다. 구체적으로는 Headless Ghidra 환경에서 대상 프로그램을 대상으로 스크립트를 실행하였다.

Ghidra는 스크립트 수준에서 비교적 쉽게 에뮬레이션을 수행할 수 있도록 EmulatorHelper를 제공한다. EmulatorHelper는 프로그램 메모리 적재, 초기 레지스터 상태 설정, 메모리 오류 처리 등 에뮬레이션에 필요한 기본 실행 환경을 자동으로 구성해 주는 보조 클래스이다. 그러나 EmulatorHelper가 기본적으로 생성하는 에뮬레이터는 기존 체계의 DefaultEmulator이며, 사용자에게 제공되는 `run()`과 `step()` 인터페이스는 명령어 단위 실행을 중심으로 설계되어 있다. 내부적으로는 각 명령어가 P-code로 변환된 뒤 Emulate 클래스의 실행 루프에서 PcodeOp 단위로 처리되지만, 단일 PcodeOp을 실행하는 메서드는 외부에서 직접 호출할 수 없도록 구현되어 있다. 따라서 EmulatorHelper만을 사용할 경우 명령어 실행 도중 개별 PcodeOp을 관찰하거나, 특정 PcodeOp에서 주변장치 모델을 호출하는 방식의 세밀한 후킹을 구현하기 어렵다.

이러한 이유로 본 구현에서는 Ghidra의 PcodeEmulator 계열을 직접 활용하였다. PcodeEmulator 자체는 공개 클래스이며, 여기서 생성되는 PcodeThread는 `stepInstruction()`뿐 아니라 `stepPcodeOp()`을 공개 메서드로 제공한다. 따라서 PcodeThread를 사용하면 명령어 단위 실행뿐 아니라 PcodeOp 단위 실행 제어도 가능하다. 다만 PcodeEmulator를 단순히 직접 생성하면 EmulatorHelper가 제공하던 프로그램 메모리 적재, 초기 레지스터 설정, 메모리 오류 처리 등의 초기 실행 환경을 별도로 연결해야 한다.

본 연구에서는 이러한 초기화 과정을 다시 구현하지 않고, EmulatorHelper가 제공하는 EmulatorConfiguration을 재사용하였다. 구체적으로는 EmulatorHelper를 생성한 뒤 이를 인자로 AdaptedEmulator를 구성하고, Java reflection을 사용하여 AdaptedEmulator 내부의 `newPcodeEmulator(EmulatorConfiguration)` 메서드를 호출하였다. AdaptedEmulator는 기존 Emulator 인터페이스와 PcodeEmulator 구현을 연결하기 위해 Ghidra에서 제공하는 전환용 클래스이다. 외부에는 기존 에뮬레이터 인터페이스처럼 보이지만,

내부적으로는 PcodeEmulator 기반으로 명령어를 실행한다. 또한 내부의 AdaptedPcodeEmulator는 EmulatorHelper가 제공하는 프로그램 메모리 이미지와 메모리 오류 처리 방식을 사용하므로, Ghidra 프로그램에 적재된 펌웨어 상태를 그대로 활용할 수 있다. 따라서 이 경로를 사용하면 Ghidra가 구성한 프로그램 메모리 적재 방식을 유지하면서도, 실행 루프에서는 PcodeThread.stepPcodeOp()을 통해 PcodeOp 단위 후킹을 수행할 수 있다.

4.3 펌웨어 초기 실행 환경 구성

앞 절의 P-code 실행 구조를 사용하더라도, 실제 펌웨어를 진입점부터 안정적으로 실행하기 위해서는 Ghidra에 가져온 프로그램 상태와 대상이 기대하는 초기 실행 상태를 맞추는 과정이 필요하다. 본 절에서는 RANDEV OBC 펌웨어를 대상으로 수행한 초기 실행 환경 구성을 설명한다. 이러한 보정은 PcodeOp 단위 실행 구조와는 별개로, 특정 펌웨어 이미지를 실제 부팅 흐름에 가깝게 실행하기 위한 대상 의존 전처리에 해당한다.

에뮬레이션의 시작 주소는 ELF 헤더에 기록된 진입점을 기준으로 설정하였다. 이를 통해 특정 펌웨어 이미지의 시작 주소를 스크립트 내부에 고정하지 않고, Ghidra에 가져온 프로그램의 적재 정보와 진입점 정보를 이용하여 초기 프로그램 카운터를 결정하도록 하였다.

또한 실제 펌웨어가 기대하는 초기 실행 상태를 맞추기 위해 일부 아키텍처 의존 레지스터 값을 명시적으로 설정하였다. 예를 들어 AVR32A 데이터시트와 QEMU의 실행 상태를 기준으로, RANDEV 펌웨어가 감독자 모드(supervisor mode)에서 시작하도록 상태 레지스터의 모드 비트를 설정하였다. 이는 Ghidra 에뮬레이터의 오류라기보다는, 분석 도구가 자동으로 제공하지 않는 대상 초기화 상태를 에뮬레이션 시작 시점에 보정한 것이다.

에뮬레이션 시작 전 Ghidra 프로그램의 메모리 이미지도 일부 보정하였다. RANDEV 펌웨어의 .data 영역은 가상 메모리 주소(VMA)와 적재 메모리 주소(LMA)가 서로 다르며, 시작 코드인 .stext는 적재 메모리 주소에 있는 초기화 데이터를 가상 메모리 주소로 복사한다. 그러나 Ghidra에 ELF 파일을 가져오면 초기화된 .data 값이 이미 가상 메모리 주소에 적재되어 있고, 적재 메모리 주소 위치에는 같은 값이 존재하지 않을 수 있다. 이 상태에서 .stext를 그대로 실행하면 적재 메모리 주소의 비어 있는 값을 가상 메모리 주소로 복사하여 .data 영역이 0으로 덮이는 문제가 발생한다. 이를 피하기 위해 별도 Ghidra 스크립트로 가상 메모리 주소의 .data 블록을 읽어 적재 메모리 주소 위치에 .data_lma 블록을 생성한 뒤, 원래 진입점부터 시작 코드가 정상적으로 실행되도록 하였다.

또한 정확한 실행 및 메타데이터 보장을 위해 일부 리스팅을 사전에 정리하였다. Ghidra가 포인터 테이블을 명령어로 해석하거나, 반대로 필요한 명령어를 함수 내부 코드로 만들지 못한 경우가 있어, 해당 주소 구간을 지운 뒤 포인터 데이터를 다시 생성하고 필요한 주소를 수동으로 역어셈블하였다. 이 작업은 에뮬레이터의 동작을 바꾸기 위한 것이 아니라, 에뮬레이션 전후의 제어 흐름 메타데이터를 비교할 때 함수 경계와 코드/데이터 구분이 불필요한 오차를 만들지 않도록 하기 위한 전처리이다.

4.4 P-code 후킹을 통한 MMIO 장치 연결

펌웨어와 주변장치 간의 MMIO 통신은 메모리 접근이 발생하는 PcodeOp에 대한 후킹을 통해 처리하였다. PcodeOp 단위 실행 중 LOAD 또는 STORE가 나타나면 해당 PcodeOp의 대상 메모리 주소를 검사한다. 접근 주소가 MMIO 주소 범위에 속하면 일반 메모리 접근으로 처리하지 않고, 해당 주소가 속한 MMIO 장치와 레지스터를 찾은 뒤 장치 모델의 read 또는 write 동작으로 전달한다.

그림 4.1와 같이, MMIO 모델은 MmioDevice와 MmioRegion의 계층 구조로 구성되어 있다. 하나의 주변장치는 MmioDevice로 표현되며 MMIO 기준 주소, 주소 범위의 크기, 인터럽트 그룹, 레지스터 테이블을 가진다. 이때 주변장치 내부에 반복되는 하위 구조가 존재하는 경우 이를 하위 MmioRegion으로

그림 4.1: RANDEV 주변장치 모델 구조도 (추가 예정)

분리한다. 예를 들어 PDCA의 레지스터 메모리 맵에는 각 DMA 채널마다 동일한 레지스터 블록(register block)이 존재하는데, 이를 PDCAChannel이라는 MmioRegion으로 분리한 뒤 PDCA 내부에 여러 개의 PDCAChannel을 만드는 식이다. MmioRegion 역시 하위 MmioRegion을 가질 수 있으며, 각 MmioRegion은 최상위 MmioDevice의 레지스터 테이블에 자신의 오프셋을 기준으로 레지스터를 등록한다. 이를 통해 MmioDevice에 대한 메모리 접근이 발생하였을 때, 각 MmioRegion을 하나하나 탐색하지 않고 곧바로 어느 레지스터에 대한 접근인지 결정할 수 있다.

4.5 타이머 및 인터럽트

큐브셋의 주변장치 중 TC는 내부 클럭에 따라 주기적으로 카운터 값을 증가시키며, 이는 INTC와 함께 주기적인 인터럽트를 발생시켜 펌웨어의 문맥 전환이 일어날 수 있게 한다.

TC 모델은 이를 반영하기 위해 독립적인 클럭 스레드를 사용하며, INTC와 결합하여 인터럽트를 발생 시키도록 하였다. 구체적으로 TC의 클럭 스레드는 내부 카운터 값을 증가시키면서 RC 비교 및 오버플로 조건을 확인한다. 둘 중 하나라도 만족하면 TC 상태 레지스터의 인터럽트 플래그를 설정하고, 연결된 INTC에 인터럽트 요청을 전달한다. INTC는 인터럽트 그룹과 각 라인별 대기 비트를 관리하며, 우선순위 레지스터 값을 기준으로 현재 처리 가능한 가장 높은 우선순위의 인터럽트 핸들러 오프셋을 저장한다.

인터럽트를 반영하기 위해 주 에뮬레이션 루프에서는 각 명령어 실행 후 현재 처리 가능한 인터럽트가 있는지 확인한다. 이때 INTC에 들어온 인터럽트가 존재하고 해당 인터럽트가 마스크된 상태가 아니라면 인터럽트에 진입한다. 인터럽트 진입은 데이터시트의 내용에 따라 현재 실행 문맥의 주요 레지스터 및 PC, SR을 스택에 저장하고, SR의 모드 비트와 인터럽트 마스크 비트를 갱신한 뒤 EVBA 기반 핸들러 주소를 계산하여 PC를 이동시키는 방식으로 구현하였다.

4.6 사용자 연산(userop) 처리

Ghidra의 Sleigh 명세는 대부분의 명령어에 대한 PcodeOp 변환 및 실행을 지원하지만, 일부 특수 명령어 또는 시스템 수준 명령어의 경우 userop이 포함된 형태로 변환된다. 예를 들어 SCALL(Supervisor call) 명령어의 경우 Sleigh 명세에서는 아래와 같이 표시된다.

```
:SCALL is op0_16=0xd733 {
    SupervisorCallSetup();
    LR = inst_next;
    tmp:4 = EVBA + 0x100;
    call [ tmp ];
}
```

위 코드의 SupervisorCallSetup이 userop이며, 이는 Ghidra 내부적으로 실행되지 않고 사용자가 직접 구현해야 하는 종류의 PcodeOp이다.

이를 위해 PcodeEmulator에는 PcodeUseropLibrary 필드가 있으며, 각 userop에 대해 실행될 메서드를 구현한 뒤 주입하였다. SCALL 및 RETE, RETS, SLEEP 등의 명령어의 P-code 변환 결과가 userop을 포함하므로, 각각에 대한 메서드를 직접 구현하였다.

4.7 I2C 장치 연결

실제 하드웨어에서 OBC와 주변장치 간의 I2C 통신은 TWIM 및 PDCA와의 상호작용을 통해 이루어진다. PDCA는 메모리와 TWIM의 THR/RHR(transmit/receive holding register) 간의 DMA 기반 전송을, TWIM은 THR/RHR과 주변장치 간의 실제 통신을 담당한다. 이러한 통신 방식을 에뮬레이터의 TWIM 및 PDCA 모델에도 반영하였다.

펌웨어가 TWIM 레지스터에 명령을 기록하면 TWIM 모델은 슬레이브 주소, 읽기/쓰기 비트, 전송 길이, START/STOP 플래그를 해석하여 I2C 통신을 수행할 수 있는 상태가 된다. I2C 장치에 값을 쓰는 경우 TWIM의 THR(transmit holding register)로 전달된 바이트를 해당 슬레이브 주소의 I2C 장치로 전달하고, 읽어오는 경우 해당 장치에게서 응답을 받아 TWIM의 RHR(receive holding register)에 저장한다.

PDCA의 경우 펌웨어가 PDCA 채널에 대상 주소와 주변장치, 통신 횟수 및 방향 등을 설정하면 각 채널에서 통신을 시작한다. PSR(peripheral select register)은 통신 대상 주변장치를 나타내며, TWIM의 수신/송신 준비 상태를 확인한 뒤 메모리와 TWIM 레지스터 사이의 바이트, 하프워드, 워드 단위 전송을 수행한다.

각 I2C 장치는 I2CDevice 추상 클래스를 상속하여 구현하였다. tx 메서드는 OBC가 장치로 보내는 명령 또는 페이로드 바이트를 처리하고, rx는 장치가 OBC로 반환할 응답 바이트를 제공한다. 또한 START_SEND, START_RECV, FINISH, NACK 이벤트를 통해 트랜잭션 시작과 종료 시점의 상태 초기화, 명령 확정, 패킷 큐 갱신 등을 수행한다. 현재 구현에는 EPS, UTX, VRX, HSTX, ADCS, UVANT 등의 장치 모델이 포함되어 I2C 통신을 수행하도록 구현되어 있다.

VRX/UTX 구현 이후에는 EPS, UVANT, ADCS, HSTX 장치 모델을 추가하였다. 각 장치의 응답은 QEMU 에뮬레이터 구현과 RANDEV 소스 코드를 함께 참고하여 구성하였다. ADCS는 내부 로직이 거의 없는 골격(skeleton) 형태였으므로 에뮬레이터에서도 동일하게 골격 형태로 구현하였다. UVANT의 경우 QEMU 구현에서 응답 길이만 지정되고 응답 버퍼가 갱신되지 않는 명령이 있어, 소스 코드와 QEMU 구현을 함께 참조하여 명령별 응답을 별도로 설정하였다.

4.8 TCP 클라이언트를 통한 RF 통신

위성과의 RF 통신 흐름을 재현하기 위해 TCP 기반의 외부 클라이언트와 에뮬레이터 내부 RF 모듈 시뮬레이터를 연결하였다. 에뮬레이터는 RF 모듈 시뮬레이터의 스레드를 실행하여 TCP 서버를 열고, 외부 클라이언트의 접속을 대기한다. 클라이언트가 원격 측정 명령 또는 RF 패킷을 전송하면 시뮬레이터는 이를 수신하여 VRX의 패킷 큐에 적재하며, 이후 펌웨어가 I2C를 통해 VRX로부터 명령을 받아 오면 해당 명령을 수행하게 된다.

반대 방향으로, 펌웨어가 프레임 송신 명령을 수행하면 UTX 장치 모델은 I2C 통신으로 전달된 페이로드를 패킷 큐에 저장한다. RF 모듈 시뮬레이터는 이 큐를 감시하다가 대기 중인 패킷이 생기면 TCP 클라이언트로 전송한다. 즉 TCP 클라이언트는 지상국 또는 외부 통신 상대처럼 동작하고, 에뮬레이터 내부의 UTX/VRX 장치는 펌웨어가 바라보는 RF 송수신 장치처럼 동작한다.

RF 모듈 시뮬레이터 구현 후에는 TCP 클라이언트가 보낸 원격 측정 명령이 VRX 패킷 큐를 통해 OBC로 전달되는지 확인하였다. 바이트 단위 메모리 접근 구현 이후 `vrx_get_frames` 내부의 두 번의 `gs_i2c_master_transaction`이 정상 동작하였고, `vrx_get_frames` 및 `vrx_get_frame`에서 VRX와의 I2C 통신이 이루어지는 것을 확인하였다. 이후 VRX에서 받은 프레임이 명령 파싱을 거쳐 `randev.sys_execute_cmd`로 전달되도록 구성하였다.

원격 측정 패킷은 처음에는 전체 원격 측정 명령 51개를 한 번에 전송하는 방식으로 구성하였다. 그러나 `vrx_get_frames()` 결과가 40개 이상이면 펌웨어가 명령을 처리하지 않는 흐름이 있어, 명령을 30개 단위로 나누어 전송하도록 수정하였다. 각 묶음의 응답이 모두 도착했을 때 다음 묶음을 보내도록 TCP

클라이언트를 구성하였고, `vr_x.remove_frame` 직후 명령은 응답을 요구하지 않도록 예외 처리하였다.

4.9 동적 제어 흐름 기록

본 연구에서는 에뮬레이션 중 관찰된 동적 제어 흐름을 별도 파일로만 저장하지 않고, Ghidra 프로그램의 메타데이터에 직접 반영하였다. 이는 에뮬레이션 결과를 이후의 정적 분석 과정에서 곧바로 사용할 수 있도록 하기 위한 것이다. 특히 간접 호출이나 간접 분기의 경우, 정적 분석 단계에서는 대상을 확인하기 어렵지만 실제 실행 중에는 분기 이후의 PC를 통해 대상 주소를 확인할 수 있다. 본 구현은 이러한 정보를 Ghidra 참조와 북마크 형태로 기록한다.

동적 제어 흐름 기록은 에뮬레이션 대상 프로그램이 `.emul` 접미사를 가진 경우에만 활성화된다. 이후 각 명령어 실행이 끝나면 실행 전 명령어와 실행 후 PC를 비교한다. 이때 해당 명령어의 FlowType을 확인하여 계산된 호출(computed call) 또는 계산된 분기(computed jump)인지 판단하고, 순차 실행(fallthrough)으로 진행된 경우를 제외한 뒤 실제 대상 주소를 기록 대상으로 삼는다.

기록 대상 흐름이 확인되면 먼저 출발지 명령어가 포함된 함수와 대상 주소가 포함된 함수를 확인한다. 출발지 함수가 없거나, 대상이 출발지와 같은 함수 내부에 머무르는 경우에는 호출 그래프 보강에 직접적인 의미가 작으므로 참조 추가 대상에서 제외한다. 새로운 함수 간 흐름으로 판단되면 Ghidra ReferenceManager를 통해 출발지 명령어 주소에서 대상 주소로 `COMPUTED_CALL`, `CONDITIONAL_COMPUTED_CALL`, `COMPUTED_JUMP`, `CONDITIONAL_COMPUTED_JUMP` 중 적절한 참조를 추가한다. 이미 동일한 흐름 참조가 존재하는 경우에는 중복으로 추가하지 않는다.

또한 간접 제어 흐름 위치가 에뮬레이션 중 한 번 이상 확인되었음을 표시하기 위해 북마크를 함께 기록한다. 북마크 종류는 `DynamicFlow`, 범주는 `IndirectFlowCovered`로 설정하였으며, 주석에는 흐름이 기록된 이유와 대상 주소를 저장한다. 동일한 출발지 명령어에서 여러 대상이 관찰될 경우에는 기존 북마크 주석에 새로운 대상 정보를 추가한다. 이후 호출 그래프 분석 단계에서는 이 북마크를 이용하여 간접 제어 흐름 위치 확인 비율을 계산하고, 참조 정보를 이용하여 새롭게 확인된 간선을 호출 그래프에 반영한다.

`userop`에 의해 발생한 제어 흐름도 별도로 처리하였다. 일부 AVR32 명령어는 Sleigh 명세에서 `CALLOTHER`를 포함하는 `userop`으로 표현되며, 이 과정에서 일반적인 계산 흐름 형태로만은 대상이 충분히 기록되지 않을 수 있다. PcodeOp 실행 중 `CALLOTHER`를 만나면 실행 후 PC를 기준으로 `userop` 흐름을 기록한다. Supervisor call 계열은 계산된 호출로, 그 외 `userop` 기반 PC 변화는 계산된 분기로 기록하여 Ghidra 참조에 반영한다.

에뮬레이션이 종료되면 기록된 참조와 북마크를 프로그램 메타데이터에 확정한다. 이렇게 반영된 메타데이터는 5장에서 수행한 호출 그래프 도달 가능성 분석 및 간접 제어 흐름 확인 비율 계산의 입력으로 사용된다. 즉 본 절의 구현은 에뮬레이터가 관찰한 실행 정보를 Ghidra 정적 분석 결과에 반영하는 핵심 연결 지점이다.

제 5 장 실험 및 결과

이 장에서는 제안한 큐브셋 펌웨어 에뮬레이터가 RANDEV OBC 펌웨어를 실제로 실행할 수 있는지, 그리고 실행 과정에서 관찰한 동적 정보를 Ghidra 프로그램 메타데이터에 반영하여 정적 분석 범위를 확장할 수 있는지를 평가한다. 이를 위해 다음 두 가지 연구 질문을 설정하였다.

- 연구 질문 1: 제안한 에뮬레이터는 RANDEV OBC 펌웨어를 정상적으로 실행할 수 있는가?
- 연구 질문 2: 제안한 에뮬레이터는 Ghidra 프로그램 메타데이터를 보강하여 정적 분석 결과를 확장할 수 있는가?

5.1 실험 환경

실험은 Ghidra에 가져온 RANDEV OBC 펌웨어를 대상으로 수행하였다. 모든 실험은 8 vCPU와 32GB RAM이 장착된 Ubuntu 22.04 장비에서 진행하였으며, 에뮬레이터는 Headless Ghidra 환경에서 Ghidra 스크립트 형태로 실행하였다.

펌웨어 실행 및 원격 측정 응답을 검증하기 위해 TCP 클라이언트를 사용하였다. TCP 클라이언트는 외부 통신 상대의 역할을 하며, 에뮬레이터 내부 RF 모듈 시뮬레이터를 통해 VRX로 원격 측정 패킷을 전달한다. 이후 펌웨어가 명령을 처리하고 UTX를 통해 응답 패킷을 송신하면, TCP 클라이언트가 이를 다시 수신한다. TCP 클라이언트로 전송한 원격 측정 패킷은 RANDEV 펌웨어가 명령 핸들러 수준에서 외부 명령어로 처리하는 I2C 장치 명령을 대상으로 구성하였다. 이는 I2C 장치 자체에는 존재하는 명령이라 하더라도, 펌웨어 단계에서 외부 원격 측정 명령으로 처리하지 않는 경우를 실험 대상에서 제외하기 위한 기준이다.

메타데이터 보강 효과를 평가하기 위해서는 에뮬레이션 전의 Ghidra 프로그램과, 에뮬레이션 중 관찰한 제어 흐름 정보를 참조 및 북마크 형태로 반영한 Ghidra 프로그램을 비교하였다. 실행 시나리오는 원격 측정을 전송하지 않고 5M 명령어를 실행하는 경우와, EPS, UVANT, ADCS, HSTX 등 I2C 장치 원격 측정 패킷을 전송한 뒤 5M 명령어를 실행하는 경우를 중심으로 구성하였다.

5.2 펌웨어 실행 및 원격 측정 응답 검증

첫 번째 연구 질문을 평가하기 위해, 제안한 에뮬레이터가 RANDEV OBC 펌웨어를 지정한 실행 시나리오에서 중단 없이 실행하고 외부 원격 측정 명령에 대한 응답을 생성하는지 확인하였다. 본 실험에서 정상 실행은 지정한 명령어 예산 동안 비정상 종료나 진행 불능 상태에 빠지지 않고, TCP 클라이언트가 전송한 원격 측정 명령에 대해 UTX를 통해 응답 패킷을 수신하는 것으로 정의하였다.

에뮬레이션 과정에서는 FreeRTOS 기반 작업(task)들이 타이머 인터럽트와 스케줄러에 의해 번갈아 실행되는지 확인하기 위해 작업별 실행 로그를 수집하였다. 수집된 로그에서는 유휴 작업(idle task)에 고정되지 않고, 각 작업이 지연 및 인터럽트에 따라 준비 상태로 복귀하여 실행되는 것을 확인하였다. 이는 타이머, INTC, supervisor call, 문맥 전환과 관련된 에뮬레이션이 펌웨어 실행에 필요한 수준으로 동작함을 보여준다.

[task execution log excerpt 추가 예정]

표 5.1: 원격 측정 명령 응답 비교 결과

장치	비교 대상 명령	일치	불일치
VRX/UTX	11	11	0
EPS	8	7	1
ADCS	9	9	0
HSTX	15	15	0
전체	43	42	1

원격 측정 응답 검증에서는 TCP 클라이언트가 보낸 명령이 VRX를 통해 OBC로 전달되고, 펌웨어의 명령 핸들러를 거쳐 UTX로 다시 송신되는지 확인하였다. 대표적으로 `vrx.get_telemetries` 명령을 전송했을 때, 펌웨어가 생성한 응답 패킷이 TCP 클라이언트에서 정상적으로 수신되었다.

Sent(7): 00 00 00 00 29 1A 00

Recv(235): 29 1A 41 41 42 42 ...

또한 동일한 원격 측정 명령에 대해 QEMU 기반 실행 결과와 Ghidra 기반 에뮬레이터의 응답을 비교하였다. QEMU 쪽 구현이 스텝 수준인 UVANT를 제외하고 총 44개의 원격 측정 명령을 전송하였으며, 이 중 VRX frame 제거 동작으로 인해 응답이 발생하지 않는 1개 명령을 제외한 43개 응답을 비교 대상으로 삼았다. 비교는 동일한 명령에 대해 TCP 클라이언트가 수신한 응답 패킷이 일치하는지 여부를 기준으로 수행하였으며, 장치별 결과는 표 5.1에 나타내었다.

표 5.1에서 볼 수 있듯이 전체 43개 응답 중 42개가 일치하여 97.7%의 일치율을 보였다. EPS가 1개 명령에서 차이를 보였고, 나머지 장치들의 경우 VRX/UTX와 ADCS는 모든 비교 대상 명령에서 동일한 응답을 반환하였다. 불일치가 발생한 사례의 원인으로는 펌웨어 쪽 구현 문제로 추정되는데 세부적으로는 5.4절에서 따로 분석한다.

이는 Ghidra 기반 에뮬레이터가 대부분의 원격 측정 명령에 대해 QEMU와 동일한 외부 응답을 재현함을 보여준다.

5.3 Ghidra 프로그램 메타데이터 보강

두 번째 연구 질문을 평가하기 위해, 에뮬레이션 전후의 Ghidra 프로그램 메타데이터를 비교하였다. 본 연구에서는 동적 실행 정보를 별도의 그래프로만 관리하지 않고, 에뮬레이션 중 관찰한 제어 흐름 정보를 Ghidra 참조와 북마크로 기존 프로그램에 반영하였다. 이후 동일한 정적 호출 그래프 생성 절차를 다시 수행하여, 메타데이터 보강이 정적 분석 결과에 미치는 영향을 확인하였다.

평가에서는 두 가지 지표를 사용하였다. 첫째, `_start`를 시작점으로 하였을 때 도달 가능한 함수의 수를 측정하였다. RANDEV 펌웨어의 실제 초기 실행 흐름은 `_start` -> `_stext` -> `main` 순서로 이어지므로, 도달 가능성 계산에서도 `_start`를 시작점으로 사용하였다. 둘째, 간접 호출 또는 간접 분기 위치 중 에뮬레이션을 통해 실제 대상이 한 번 이상 확인된 위치의 비율을 측정하였다.

도달 가능성 계산에서는 실험 시나리오와 직접 관련이 낮은 콘솔 명령 계열 함수를 제외하였다. 구체적으로 `cmd_`, `gs_log_cmd` 등 명령 콘솔과 관련된 접두사를 갖는 함수, 접두사만으로 식별되지 않는 일부 명령 핸들러, 그리고 이들 제외 함수에서만 정적으로 도달 가능한 함수들을 제외하였다. 이를 통해 펌웨어 실행 시나리오와 무관한 명령 콘솔 경로가 결과를 과도하게 희석하지 않도록 하였다.

표 5.2는 에뮬레이션으로 보강한 메타데이터를 반영한 뒤 도달 가능성이 증가한 결과를 보여준다. 평가 대상 함수 기준으로 도달 가능 함수 비율은 78.0%에서 83.1%로 5.1%p 증가하였다. 그러나 이 값은 기존에

표 5.2: 에뮬레이션 전후 도달 가능 함수 변화

지표	에뮬레이션 전	에뮬레이션 후	변화
도달 가능 함수	744/953 (78.0%)	792/953 (83.1%)	+48 (+5.1%p)
도달 불가능 함수	209/953 (22.0%)	161/953 (16.9%)	-48 (-5.1%p)
신규 도달 가능 함수	-	48/209	23.0%

표 5.3: 간접 제어 흐름 대상 확인 결과

지표	값
평가 대상 간접 제어 흐름 위치	58
대상이 관찰된 위치	16
간접 제어 흐름 위치 확인 비율	27.6%

이미 도달 가능했던 함수까지 분모에 포함하므로, 에뮬레이션을 통해 새로 보강된 부분의 효과를 상대적으로 작게 보이게 한다. 본 연구에서 중요한 변화는 기존 정적 분석에서 도달 불가능으로 남아 있던 함수들이 새롭게 호출 그래프에 연결되었는지 여부이다. 이 관점에서 보면 도달 불가능 함수 수는 209개에서 161개로 감소하였고, 기존 도달 불가능 함수 중 48개, 즉 23.0%가 새롭게 도달 가능한 것으로 분류되었다. 참고로 필터링 전 전체 함수 기준으로도 도달 가능한 함수는 744개에서 792개로 48개 증가하였다.

간접 제어 흐름 위치의 경우, 전체 66개 위치 중 실험 대상에서 제외한 8개를 제거하여 58개 위치를 평가 대상으로 삼았다. 이 중 16개 위치에서 에뮬레이션을 통해 실제 대상이 한 번 이상 확인되었으며, 이는 27.6%의 간접 제어 흐름 위치 확인 비율에 해당한다. 이러한 결과는 제한한 에뮬레이터가 실행 중 관찰한 동적 제어 흐름 정보를 Ghidra 프로그램 메타데이터에 반영함으로써, 기존 정적 분석만으로 확인하기 어려웠던 함수 도달성과 간접 제어 흐름 대상을 보강할 수 있음을 보여준다.

5.4 확인된 불일치 사례

본 연구의 주된 목표는 버그 탐지가 아니지만, 에뮬레이터를 구현하고 QEMU 및 펌웨어 소스 코드와 동작을 비교하는 과정에서 Ghidra 명세, AVR32 데이터시트, 기존 QEMU 장치 모델, 펌웨어 내부 로직 사이의 불일치를 확인할 수 있었다. 이 절에서는 단순한 구현 실수나 장치 모델 조정 과정에서 발생한 문제를 제외하고, 기존 명세와 구현 사이의 차이 또는 펌웨어 소스 코드의 문제로 판단되는 사례를 정리한다.

Ghidra AVR32 Sleigh 명세에서는 세 가지 문제가 확인되었다. 첫째, CPC 명령어의 C 플래그 계산이 AVR32 데이터시트의 캐리(carry) 계산과 달라, 뒤따르는 분기의 조건 판정이 잘못되고 무한루프가 발생하는 사례가 있었다. 둘째, 조건부 ST.B 명령어의 변위(displacement)가 바이트 단위가 아니라 워드 단위처럼 해석되어, 실제로는 스택 버퍼에 저장되어야 할 값이 저장된 LR 위치에 저장되는 문제가 확인되었다. 셋째, MOV PC, Rn 및 조건부 반환(return) 계열 명령어에서 PC 레지스터 값은 갱신되지만, BRANCHIND 또는 RETURN과 같은 분기용 PcodeOp이 생성되지 않아 실제 프로그램 카운터가 이동하지 않는 사례가 있었다.

AVR32 데이터시트에서도 RETE 설명과 관련된 불일치가 확인되었다. 데이터시트에 따르면 CPU가 인터럽트 모드에 진입할 때에는 기존 SR 값을 스택에 저장한 뒤 현재 SR의 모드 비트를 인터럽트 모드로 변경한다. 그러나 RETE 설명에서는 스택에서 복구한 SR 값을 기준으로 추가 레지스터 pop 여부를 결정하는 것처럼 기술되어 있다. 이 경우 인터럽트 진입 시 push한 레지스터 집합과 RETE에서 pop하는 레지스터 집합이 서로 달라질 수 있다. 따라서 실제 동작을 기준으로 스택에서 복구한 SR이 아니라 RETE 실행 시점의

SR 모드를 기준으로 레지스터 복원 여부를 결정해야 하는 것으로 판단하였고, 해당 내용을 데이터시트 설명 오류 가능성이 있는 사례로 별도 보고하였다.

펌웨어 내부 로직 자체의 문제로는 두 가지 사례가 확인되었다. 첫째, `vrx_get_uptime` 명령에서는 VRX 모델이 `0xCAFEBAFE`를 반환하도록 구성되어 있었으나, 실제 TCP 클라이언트에서는 `29 40 BE BE BE BE ...` 형태의 응답이 수신되었다. 이 사례는 QEMU와 Ghidra 모두 같은 응답을 반환하였기 때문에 표 5.1의 불일치로 나타나지는 않았지만, 장치 모델의 의도한 반환값과 펌웨어 응답을 비교하는 과정에서 확인되었다. 원인 분석 결과, `TRXUV_GET_UPTIME` 처리에서 `memcpy`가 아니라 `memset`이 호출되어 첫 바이트인 `0xBE`가 반복 복사되는 것으로 확인되었다. 둘째, 표 5.1에서 나타난 EPS 응답 불일치는 두 번째 `eps_get_status` 응답에서 발생하였으며, Ghidra 실행에서는 timeout 상태값인 `FB`가 반환된 반면 QEMU 실행에서는 정상 응답값인 `AA AA`가 반환되었다. 문제 원인으로는 EPS 명령이 delay가 있는 경로에서 송신과 수신을 분리해 수행하기 때문에 그 사이에 인터럽트와 문맥 전환이 발생하면서 downlink manager와 uplink manager의 상태가 어긋나는 것으로 분석되었다. 실제로 Ghidra 실행에서 해당 delay를 제거한 경로로 강제했을 때 정상값이 반환되었고, QEMU에서는 실행 속도가 빨라 두 작업이 동시에 ready 상태가 되는 상황이 상대적으로 적어 같은 문제가 드러나지 않은 것으로 추정된다.

이러한 사례들은 에뮬레이터를 구현하고 검증하는 과정에서 Ghidra 명세, 하드웨어 데이터시트, QEMU 장치 모델, 펌웨어 소스 코드 사이의 불일치를 확인할 수 있었음을 보여준다.

제 6 장 논의 및 향후 연구 방향

6.1 다른 펌웨어로의 확장 가능성

본 에뮬레이터는 Ghidra P-code 실행 엔진을 기반으로 하므로 명령어 의미 계층에서의 아키텍처 의존성이 낮다. Ghidra가 대상 명령어 집합 구조(Instruction Set Architecture, ISA)의 Sleigh 명세를 제공한다면, 별도의 명령어 해석기를 새로 작성하지 않고도 PcodeOp 단위 실행을 재사용할 수 있다. 또한 PcodeOp 후킹, LOAD/STORE 기반 MMIO 처리, MmioDevice와 MmioRegion 계층 구조, I2CDevice 추상화, 동적 제어 흐름 기록 및 Ghidra 메타데이터 갱신 방식은 특정 ISA에 강하게 결합되어 있지 않다.

다만 실제 펌웨어 실행 가능성은 명령어 의미만으로 결정되지 않는다. 큐브셋 펌웨어는 보드별 주변장치, 인터럽트 맵, 타이머, DMA/I2C 컨트롤러, RF 모듈, 명령-응답 통신 규약에 강하게 의존한다. 따라서 새로운 대상에 본 프레임워크를 적용하려면 다음과 같은 정보를 추가로 제공하거나 모델링해야 한다.

- 펌웨어 바이너리의 메모리 배치, 진입점, LMA/VMA 복사 방식, 시작 코드 초기화 흐름
- MMIO 기준 주소, 레지스터 맵, 레지스터별 부수 효과
- TC의 클럭 소스, 비교/오버플로 조건, 인터럽트 발생 조건
- INTC의 그룹/라인, 우선순위 레지스터, 핸들러 벡터 계산 방식
- DMA/I2C 컨트롤러의 레지스터 인터페이스와 전송 순서
- I2C 슬레이브 주소, 장치별 명령-응답 통신 규약, 응답 페이로드 형식
- RF 송수신 장치 및 외부 클라이언트와 주고받는 패킷 형식
- 대상 ISA 또는 Sleigh 명세에서 userop으로 남는 명령어의 실행 의미
- RTOS 작업 추적에 필요한 TCB 배치, 스케줄러 관련 심볼 또는 오프셋

현재 구현에서도 RANDEV에 특화된 정보가 일부 남아 있다. 대표적으로 RANDEV의 MMIO 주소 범위와 주변장치 구성, PDCA/TWIM 채널 매핑, EPS/VRX/UTX/HSTX/ADCS/UVANT 등 장치별 I2C 명령 집합, 원격 측정 명령 ID와 응답 페이로드 형식은 RANDEV 보드와 펌웨어에 의존한다. 또한 분석 및 실험 과정에서 일부 펌웨어별 우회 처리가 사용되었다. 예를 들어 `vfprintf_r` 내부 실행을 우회한 처리와, VRX 프레임 처리 개수에 맞춘 원격 측정 묶음 구성은 다른 펌웨어에서는 그대로 적용된다고 보기 어렵다.

따라서 본 연구에서 주장하는 일반성은 모든 큐브셋 펌웨어를 수정 없이 즉시 실행할 수 있다는 의미가 아니다. 일반성은 Ghidra P-code 실행 계층과 주변장치 모델 연결 방식, 그리고 동적 실행 결과를 Ghidra 메타데이터에 반영하는 구조가 다른 대상에도 재사용 가능하다는 데 있다. 다른 큐브셋 펌웨어에 적용하려면 보드 기술 정보(board description), 장치 모델, 통신 규약 모델을 새로 추가해야 한다.

6.2 실행 속도 한계

현재 구현된 에뮬레이터는 QEMU 기반 에뮬레이터와 비교할 때 실행 속도가 크게 느리다. 이를 비교하기 위해 에뮬레이터 프로세스 실행 시점, TCP 클라이언트의 첫 패킷 전송 시점, 마지막 응답 수신 시점을 기준으로 실행 시간을 나누어 측정하였다. 그 결과는 표 6.1과 같다.

표 6.1: QEMU와 Ghidra 기반 에뮬레이터의 실행 시간 비교

구간	QEMU 시간(초)	Ghidra 시간(초)	속도 저하
명령 수신 준비 시간	1.003	12.015	12.0배
원격 측정 명령 처리 시간	5.941	199.924	33.7배
전체 실행 시간	6.943	211.938	30.5배

표 6.1에서 Ghidra 기반 에뮬레이터는 명령 수신 준비 단계에서 QEMU보다 약 12.0배 느렸고, 첫 원격 측정 패킷 전송 이후 마지막 응답을 수신하기까지의 명령 처리 구간에서는 약 33.7배 느렸다. 전체 실행 시간은 QEMU가 6.943초, Ghidra 기반 에뮬레이터가 211.938초로 약 30.5배의 차이를 보였다. 특히 명령 처리 구간에서 차이가 더 크게 나타난 것은 초기 구동 비용뿐 아니라, 실행 중 PcodeOp 단위 처리와 MMIO, I2C, 인터럽트 후킹 비용이 반복적으로 누적되기 때문으로 볼 수 있다.

이러한 속도 차이는 두 도구의 목적 차이에서 비롯된다. QEMU는 게스트 명령어를 TCG 중간 표현으로 변환한 뒤 변환 블록(translation block) 단위로 묶어 호스트에서 효율적으로 실행함으로써 빠른 실행을 목표로 한다. 반면 Ghidra는 고성능 전체 시스템 에뮬레이션보다는 역공학과 정적 분석을 위한 도구이며, 명령어 의미를 P-code라는 분석 가능한 중간 표현으로 유지한다. 이 때문에 실행 속도는 느리지만, PcodeOp 단위에서 MMIO 접근, userop, 간접 제어 흐름 등을 관찰하고 후킹할 수 있다.

따라서 본 연구에서의 성능 저하는 Ghidra 기반 접근이 갖는 예상 가능한 절충점으로 볼 수 있다. 본 연구의 목적은 펌웨어를 빠르게 반복 실행하는 것이 아니라, 실행 과정에서 관찰한 제어 흐름 정보를 Ghidra 프로그램 메타데이터에 반영하여 정적 분석 범위를 확장하는 것이다. 이 관점에서 실행 시간이 길어지는 한계는 존재하지만, 동일한 프로그램 데이터베이스 안에서 실행 결과와 정적 분석 메타데이터를 결합할 수 있다는 점이 본 접근의 장점이다.

6.3 향후 연구 방향

향후에는 우선 다른 큐브셋 펌웨어에도 본 프레임워크를 적용하여 일반성을 검증할 필요가 있다. 이를 위해 현재 Java 코드에 직접 포함된 대상별 정보를 설정 파일(configuration file) 또는 장치 기술 정보(device description) 형태로 분리할 수 있다. 예를 들어 MMIO 장치 이름, 기준 주소, 주소 크기, 레지스터 정보, 인터럽트 그룹/라인 매핑, I2C 슬레이브 주소, DMA 채널과 PSR 매핑, 원격 측정 명령 ID와 기대 응답 등을 별도 파일에 기술하고, 에뮬레이터 핵심부는 이를 읽어 장치들을 구성하도록 만들 수 있다. 이러한 구조를 사용하면 새로운 펌웨어에 적용할 때 Java 코드 수정량을 줄일 수 있으며, 사용자 입장에서 대량으로 수정해야 하는 부분과 공통 에뮬레이터 핵심부를 명확히 구분할 수 있다.

속도 문제는 Ghidra와 QEMU를 결합한 혼합 방식으로 완화할 수 있다. 한 가지 방향은 Ghidra가 정적 분석을 통해 간접 제어 흐름 위치, MMIO 접근 위치, userop 위치를 미리 수집하고, QEMU가 빠르게 펌웨어를 실행하면서 해당 주소에 도달했을 때만 실행 기록 또는 콜백(callback)을 발생시키는 것이다. 반대로 QEMU 실행 기록을 먼저 수집한 뒤, 이를 Ghidra 스크립트가 가져와 참조, 북마크, 호출 그래프 메타데이터로 반영하는 오프라인 처리 절차도 고려할 수 있다. 이 방식은 QEMU의 실행 속도와 Ghidra의 프로그램 메타데이터를 함께 활용할 수 있으나, 두 도구 사이의 주소 공간, 심볼, 함수 경계를 일관되게 맞추는 추가 작업이 필요하다.

또한 본 연구에서는 호출 그래프 도달 가능성과 간접 제어 흐름 대상 복원을 중심으로 평가하였지만, 이는 정적 분석 보강의 한 예시일 뿐이다. 동일한 구조는 명령 핸들러 도달 가능성, 데이터 흐름 분석(data-flow analysis), 오염 분석(taint analysis) 등으로 확장될 수 있다. 예를 들어 지상국 명령 패킷이 어떤 하위 시스템 함수까지 도달하는지, I2C 장치 응답이 분기 조건이나 작업 경로에 어떤 영향을 미치는지 분석하는 방향을

고려할 수 있다. 이를 통해 큐브셋 펌웨어의 제어 흐름뿐 아니라 명령 처리 경로와 데이터 의존 동작까지 분석 범위를 넓힐 수 있을 것이다.

제 7 장 관련 연구

7.1 펌웨어 에뮬레이션

펌웨어 에뮬레이션은 임베디드 시스템의 동작을 실제 하드웨어 없이 재현하기 위한 대표적인 방법이다. 특히 QEMU는 동적 바이너리 변환(dynamic binary translation)을 기반으로 게스트 명령어를 호스트에서 실행 가능한 형태로 변환하여 빠른 실행 속도를 제공하는 에뮬레이터로 널리 사용된다[5]. QEMU와 같은 전체 시스템 에뮬레이터는 운영체제 또는 펌웨어 이미지를 실제 하드웨어와 유사한 환경에서 실행할 수 있으며, 장치 모델을 추가하면 특정 보드의 주변장치 동작도 재현할 수 있다.

그러나 QEMU는 실행 중심의 시스템 에뮬레이터이므로, 함수 경계, 참조, 호출 그래프, 북마크와 같은 역공학 메타데이터를 자체적으로 제공하거나 관리하지 않는다. 따라서 QEMU 실행 중 관찰한 MMIO 접근이나 간접 제어 흐름 정보를 정적 분석에 활용하려면, 별도의 로그 수집 및 분석 도구와의 연동 과정이 필요하다. 본 연구에서는 Ghidra P-code 실행 엔진을 사용하여 PcodeOp 단위 후킹을 확보하고, 실행 중 관찰된 간접 제어 흐름을 Ghidra 참조와 북마크로 기록한다는 점에서 기존 QEMU 중심 접근과 차이가 있다.

7.2 위성 및 큐브셋 소프트웨어 검증

위성 및 큐브셋 소프트웨어 검증에서는 실제 하드웨어 또는 하드웨어 모델을 포함한 시험 환경이 중요하게 다루어진다. 하드웨어-인-더-루프(Hardware-in-the-loop, HIL) 및 소프트웨어-인-더-루프(Software-in-the-loop, SIL) 방식은 비행 소프트웨어를 실제 보드, 주변장치, 시뮬레이션 환경과 연결하여 지상에서 검증하는 방법이다. MOVE-II CubeSat의 HIL/SIL 시험 환경은 이러한 접근의 대표적인 사례로, 큐브셋 소프트웨어를 실제 임무 환경에 가깝게 시험하기 위한 구조를 제시한다[2].

이러한 검증 방식은 실제 하드웨어와 소프트웨어가 함께 준비된 개발 단계에서는 높은 신뢰도를 제공한다. 그러나 바이너리 펌웨어만 확보되어 있거나, 실제 보드에 반복적으로 접근하기 어려운 경우에는 HIL/SIL 환경을 그대로 구성하기 어렵다. 또한 HIL/SIL은 주로 정상 동작 검증과 통합 시험에 초점을 두므로, 역공학 도구의 호출 그래프, 참조, 북마크와 같은 메타데이터를 확장하는 목적과는 직접적으로 연결되지 않는다.

위성 소프트웨어의 보안 분석을 다룬 연구도 존재한다. Space Odyssey는 실제 위성 소프트웨어를 대상으로 실험적 보안 분석을 수행하여, 위성 소프트웨어가 일반적인 임베디드 소프트웨어와 마찬가지로 취약점 분석과 검증의 대상이 될 수 있음을 보였다[6].

본 연구 역시 위성 펌웨어를 분석 대상으로 삼지만, 취약점 탐지 자체보다는 Ghidra 기반 정적 분석을 보완하기 위한 에뮬레이션 환경 구축에 초점을 둔다. 즉 본 연구는 큐브셋 펌웨어의 명령-응답 실행과 주변장치 상호작용을 재현하고, 그 결과를 Ghidra 프로그램 메타데이터에 반영한다는 점에서 기존 검증 및 보안 분석 연구와 구분된다.

7.3 정적 분석 및 Ghidra 기반 분석

본 연구에서 사용된 Ghidra는 바이너리 역공학을 위한 공개 소프트웨어 분석 프레임워크로, 역어셈블, 역컴파일, 심볼 및 참조 분석, 함수 복원 등 다양한 정적 분석 기능을 제공한다[3]. Ghidra의 중요한 특징 중 하나는 프로세서 아키텍처별 명령어 의미를 Sleight 명세 언어로 정의하고, 이를 P-code라는 중간 표현으로 변환한다는 점이다[4]. P-code는 아키텍처 의존 명령어를 비교적 일반적인 연산 단위로 표현하므로, 다양한 아키텍처에 대해 공통적인 분석 또는 실행 기반 처리를 적용할 수 있는 기반이 된다.

기존 Ghidra 기반 정적 분석은 바이너리 내부의 명령어, 데이터 참조, 함수 경계, 호출 그래프를 복원하는 데 강점을 가진다. 그러나 펌웨어가 MMIO, 인터럽트, I2C 응답, RF 패킷과 같은 외부 실행 환경에 의존하는 경우, 정적 분석만으로는 실제 실행 중 형성되는 제어 흐름을 모두 확인하기 어렵다. 특히 간접 호출, 간접 분기, 콜백, 명령 테이블 기반 호출과 같은 흐름은 실행 시점 상태가 주어졌을 때 대상이 결정되는 경우가 많다.

본 연구는 Ghidra의 정적 분석 기능을 대체하는 것이 아니라, Ghidra 내부에서 수행한 에뮬레이션 결과를 기존 프로그램 메타데이터에 추가하여 정적 분석 범위를 확장한다. 이를 위해 Ghidra P-code 실행 엔진을 사용하고, PcodeOp 단위로 MMIO 접근과 간접 제어 흐름을 관찰한다. 관찰된 결과는 외부 파일로만 보관하지 않고 Ghidra 참조와 북마크로 반영되며, 이후 기존 호출 그래프 분석 절차에서 자연스럽게 사용된다. 따라서 본 연구는 Ghidra의 P-code 기반 분석 가능성과 펌웨어 재호스팅을 결합하여, 주변장치 의존적인 큐브셋 펌웨어의 정적 분석을 보완하는 접근으로 볼 수 있다.

제 8 장 맺음말

본 연구는 Ghidra P-code 실행 엔진을 기반으로 큐브셋 펌웨어를 실행하고, 그 결과를 Ghidra 정적 분석 메타데이터에 반영하는 아키텍처 독립적 펌웨어 에뮬레이터를 제안하였다. 제안한 에뮬레이터는 Ghidra 스크립트 형태로 구현되었으며, PcodeOp 단위 실행을 관찰하여 MMIO 접근, userop, 간접 제어 흐름을 후킹한다. 또한 RANDEV OBC 펌웨어 실행에 필요한 타이머/인터럽트, PDCA/TWIM 기반 I2C 통신, RF 명령 및 원격 측정 흐름을 장치 모델로 구성하여 실제 하드웨어 없이도 펌웨어가 기대하는 관찰 가능한 동작을 제공하였다.

구현한 에뮬레이터를 RANDEV OBC 펌웨어에 적용한 결과, 펌웨어는 지정한 실행 시나리오에서 정상적으로 실행되었고 TCP 클라이언트를 통해 전달한 원격 측정 명령에 대해 UTX를 통한 응답 패킷을 생성하였다. 또한 에뮬레이션 중 관찰된 간접 제어 흐름 대상을 Ghidra 참조와 북마크로 기록함으로써, 기존 정적 분석만으로는 확인되지 않던 호출 그래프 간선과 도달 가능 함수를 추가로 확인할 수 있었다. 평가 결과, 필터링 후 함수 기준 도달 가능 함수 비율은 78.0%에서 83.1%로 증가하였고, 간접 제어 흐름 위치 58개 중 16개에서 실제 대상을 확인하였다.

본 연구의 의의는 단순히 펌웨어를 실행하는 데 있지 않다. QEMU와 같은 고성능 에뮬레이터가 실행 속도와 시스템 에뮬레이션에 강점을 가진다면, 본 연구는 Ghidra 내부의 P-code와 프로그램 메타데이터를 활용하여 실행 결과를 정적 분석 환경에 반영한다는 점에 초점을 둔다. 이를 통해 타이머, 인터럽트, I2C 응답, RF 패킷과 같이 정적 분석만으로 다루기 어려운 실행 시점 의존 동작을 분석 과정에 반영할 수 있음을 보였다.

다만 본 연구의 구현은 아직 RANDEV에 특화된 장치 모델과 통신 규약 정보를 포함하고 있으며, 주기 정확 하드웨어 시뮬레이션을 목표로 하지 않는다. 향후에는 다른 큐브셋 펌웨어에 적용하여 프레임워크의 일반성을 추가로 검증하고, MMIO 맵, 인터럽트 맵, I2C 주소, 원격 측정 통신 규약과 같은 대상별 정보를 설정 파일 또는 장치 기술 정보 형태로 분리할 필요가 있다. 또한 Ghidra와 QEMU를 결합하여 QEMU의 실행 속도와 Ghidra의 메타데이터 관리 기능을 함께 활용하는 혼합 분석 구조도 후속 연구 방향이 될 수 있다.

결론적으로 본 연구는 Ghidra 환경에서 큐브셋 펌웨어의 실행 환경을 구성하고, 동적 실행 정보를 정적 분석 메타데이터에 반영함으로써 펌웨어 분석 범위를 확장할 수 있음을 보였다. 이는 실제 하드웨어나 완전한 소스 코드 접근이 제한적인 상황에서도 큐브셋 펌웨어의 구조와 실행 경로를 이해하기 위한 기반이 될 수 있다.

참 고 문 헌

- [1] The CubeSat Program, Cal Poly SLO, CubeSat Design Specification Rev. 14.1, 2022.
- [2] J. Kiesbye, D. Messmann, M. Preisinger, G. Reina, D. Nagy, F. Schummer, M. Mostad, T. Kale, and M. Langer, “Hardware-In-The-Loop and Software-In-The-Loop Testing of the MOVE-II CubeSat,” *Aerospace*, vol. 6, no. 12, article 130, 2019.
- [3] National Security Agency, Ghidra Software Reverse Engineering Framework, 2019.
- [4] Ghidra Project, SLEIGH: A Language for Rapid Processor Specification, 2023.
- [5] F. Bellard, “QEMU, a Fast and Portable Dynamic Translator,” in *Proc. 2005 USENIX Annual Technical Conference*, Anaheim, CA, USA, pp. 41-46, 2005.
- [6] J. Willbold, M. Schloegel, M. Vögele, M. Gerhardt, T. Holz, and A. Abbasi, “Space Odyssey: An Experimental Software Security Analysis of Satellites,” in *Proc. 44th IEEE Symposium on Security and Privacy*, 2023.
- [7] CISPA SysSec, SpaceOdyssey-QEMU-AVR32: QEMU-AVR32 OPS-SAT Emulator, 2023.

사 사

본 연구를 지도해 주신 류석영 교수님께 깊이 감사드립니다. 연구 과정에서 조언과 도움을 주신 연구실 구성원들께도 감사드립니다. 마지막으로 언제나 응원해 준 가족과 주변 사람들에게 감사의 마음을 전합니다.

약 력

이 름: 류 수 현

학 력

2024. 3. – 2026. 8. 한국과학기술원 전산학부 (석사)

경 력

2024. 3. – 2026. 8. 한국과학기술원 전산학부 석사과정

연 구 업 적

1. S. Ryu, *Ghidra P-code based architecture-independent CubeSat firmware emulator*, Master Thesis, Korea Adv. Inst. Science, Techn., Daejeon, Republic of Korea, 2026.